

BUILD SPECIFICATION / PART 1 OF 2 // v1.0 // SEPTEMBER 2026

High-Level Design

Food Delivery Platform - Enhancement

Frontend build + backend hardening for the existing Node/Express/MongoDB backend

Contents

Contents	2
1. Agent Brief (Read First)	3
2. Product Overview	3
3. Users and Key Journeys	4
4. Information Architecture	4
5. System Architecture	5
6. Technology Stack	6
7. Feature Overview	6
8. UX and Visual Direction	7
9. Data Overview	8
10. Security and Privacy	8
11. Non-Functional Requirements	8
12. Environments and Deployment View	9
13. Delivery Plan, Risks and Assumptions	9

1. Agent Brief (Read First)

This is an **enhancement** project, not a from-scratch build. A working backend already exists, hand-coded, in a local folder (Node.js/Express 5, MongoDB Atlas, JWT auth, RBAC, Cashfree payments). This HLD + LLD pair specifies (a) the backend hardening it needs and (b) an entirely new, industry-level React frontend for three roles: customer, restaurant owner, and admin.

- **Do not rewrite the backend from scratch.** Extend and harden the existing code. Section 15 (LLD) lists the exact fixes required, each tied to a real issue found in the current codebase.
- **Build the frontend from scratch** as a new `client/` app in the same repository, alongside the existing backend (moved into `server/` if it is not already isolated).
- **Initialise Git first.** The project currently has no repository. Commit the existing backend as-is before making any changes, so the "before" state is preserved in history.
- **Read order:** this HLD for context and direction, then the LLD for exact schemas (mostly unchanged), endpoints (existing + new), component specs, and the phase-by-phase plan.
- **Placeholders** (brand name, logo, sample restaurants/menu, prices) live in one config/content location and are trivial to replace later.
- **Final deliverables:** a working Git repository, README, `.env.example`, live frontend and API URLs, a seeded demo dataset (restaurants, foods, a demo customer/owner/admin login), and a handover note listing every fix applied and every placeholder to replace.

2. Product Overview

2.1 What exists today

A REST API for a food delivery marketplace: users register and log in with JWT-based auth and one of three roles (User, restaurantOwner, admin); restaurant owners manage restaurants and menus; customers browse, add to cart, check out, pay via Cashfree, and track an order through a status pipeline; admins approve owner requests. Redis caches restaurant and food listings. There is no frontend and no deployment yet.

2.2 What this project adds

Goal	What it means
A real product, not just an API	A polished, fast, three-role React frontend that makes the existing backend usable end to end
Visual quality that reads as "industry level"	A vibrant, food-forward design system used with total consistency across every screen, not just the customer-facing ones
Real-time order tracking	Replace status polling with a live order-status feed (Socket.IO) so customers and owners see updates instantly
Backend hardening	Fix the concrete issues found in the status report: order/cart transaction safety, secure cookies, image uploads, wiring the email worker, removing dead dependencies, adding tests
Operable in production	Docker, CI, structured logging, and a deployment runbook so the project can actually go live, not just run on localhost

2.3 Scope

In scope (v1 of the enhancement)	Out of scope (v1)
Customer web app: browse, cart, checkout, pay, track, history, profile, addresses Restaurant owner dashboard: restaurant profile, menu manager, incoming orders Admin dashboard: owner requests, users, restaurants overview Image upload for restaurants and food items Real-time order status via Socket.IO Backend hardening items from section 15 (LLD) Automated tests, Docker, CI, deployment	Native mobile apps Live delivery-partner tracking / rider app Geolocation-based delivery-fee or ETA calculation Ratings and reviews Multi-language / multi-currency Admin analytics dashboards beyond basic counts

3. Users and Key Journeys

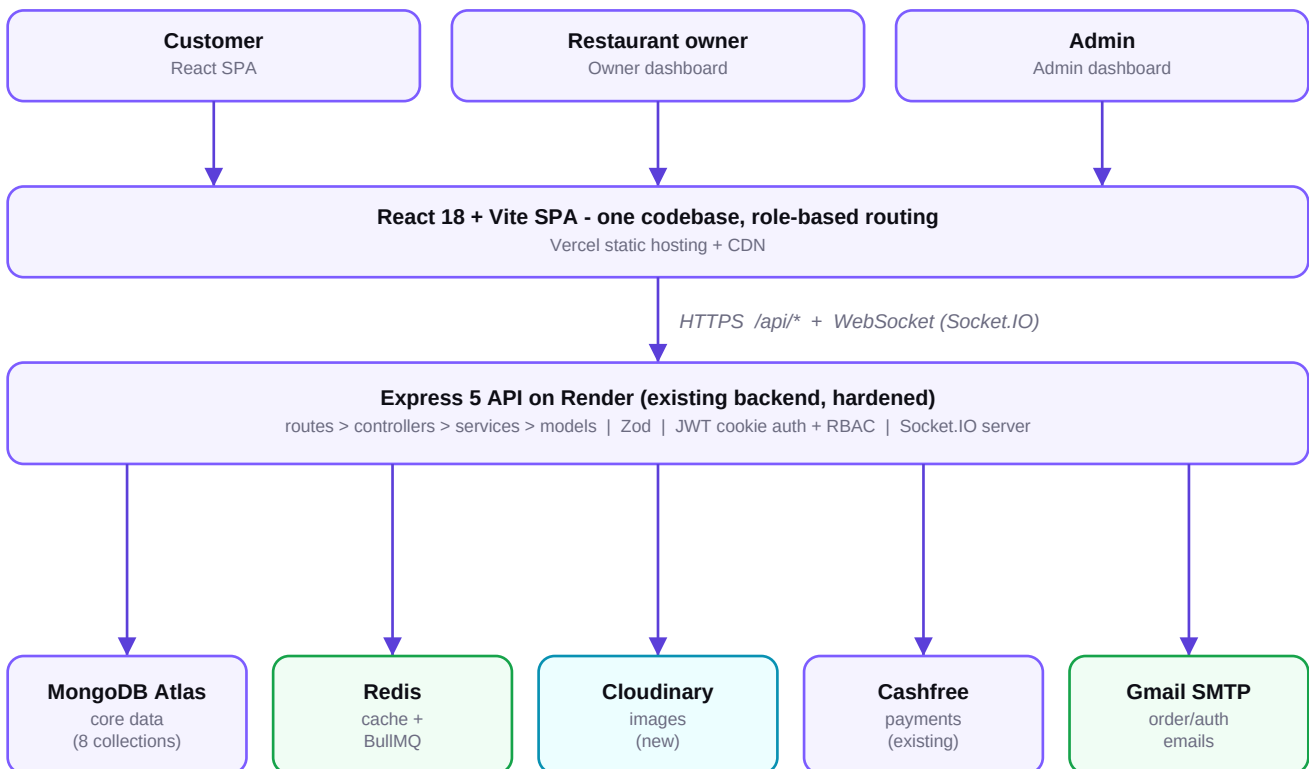
Persona	Existing backend support	Primary journey (new frontend)
Customer	Full auth, cart, checkout, order, address, payment APIs already exist	Sign up > browse restaurants > view menu > add to cart > checkout > pay > track order live > view history
Restaurant owner	Restaurant/food CRUD, incoming-orders and status-update APIs already exist	Request owner role > create restaurant > build menu with photos > receive orders in real time > update status through to delivered
Admin	Owner-request approval APIs already exist	Review pending owner requests > approve or reject > see a top-level view of users and restaurants

4. Information Architecture

Route	Page	Role	Notes
/	Home / discover	Public	Hero, cuisine categories, featured and nearby restaurants
/restaurants	Restaurant listing	Public	Search, category and open/closed filters, pagination
/restaurants/:id	Restaurant menu	Public (add to cart requires login)	Menu grouped by category, search, add to cart
/cart	Cart	Customer	Single-restaurant cart, quantity edit, price breakdown
/checkout	Checkout	Customer	Address selection, price summary, Cashfree payment
/orders	Order history	Customer	Past and active orders, filters by status
/orders/:id	Order tracking	Customer	Live status timeline via Socket.IO, cancel while eligible
/profile, /addresses	Account	Customer	Profile edit, password change, saved addresses, "become a restaurant owner" request
/login, /register	Auth	Public	JWT cookie session
/owner	Owner dashboard home	restaurantOwner	Restaurant status, today's order counts
/owner/restaurant	Restaurant profile	restaurantOwner	Edit details, open/closed toggle, cover image
/owner/menu	Menu manager	restaurantOwner	Add/edit/delete food items with image upload
/owner/orders	Incoming orders	restaurantOwner	Live queue, advance status

Route	Page	Role	Notes
/admin	Admin dashboard home	admin	Counts: users, restaurants, pending requests
/admin/owner-requests	Owner requests	admin	Approve / reject with reason
/admin/users, /admin/restaurants	Directories	admin	Read-heavy tables with search and pagination
*	404	Public	Branded not-found page

5. System Architecture



One React SPA serves all three roles behind role-based routing and route guards, talking to the existing Express API (hardened) over HTTPS and a Socket.IO channel for live order updates. Cloudinary is a new addition for image uploads; everything else in the data layer already exists and is reused as-is.

5.1 What changes in the backend vs. what stays

Stays as-is	Changes
All 8 Mongoose models Auth (JWT access/refresh, RBAC) Restaurant, food, cart, checkout, order, address, owner-request logic Cashfree payment integration Redis caching, rate limiting, Zod validation	Add Cloudinary image upload endpoint Add Socket.IO server for order-status events Wire the existing BullMQ email worker and cron into server startup Add Mongoose transactions to order placement Fix cookie <code>secure/sameSite</code> for production Remove unused <code>razorpay</code> dependency; fix <code>package.json</code> "main" Add a thin services layer under the existing controllers where logic is duplicated Add structured logging, tests, Docker, CI

6. Technology Stack

Layer	Choice	Status
Frontend framework	React 18 + Vite 5	New
Routing	React Router v6 with role-based route guards	New
Styling	Tailwind CSS 3.4 + CSS variable tokens	New
Motion	Framer Motion	New
Server state	TanStack Query v5 + Axios	New
Real-time	Socket.IO client, subscribed per order / per restaurant room	New (server side is also new)
Forms	React Hook Form + Zod resolvers (mirrors backend Zod schemas)	New
Maps/eta	None in v1 (static delivery fee, as today)	Unchanged
Runtime	Node.js (existing), Express 5	Existing, hardened
Database	MongoDB Atlas + Mongoose	Existing, unchanged
Cache / queue backend	Redis (existing) + BullMQ (existing, to be wired up)	Existing, wired up
Real-time server	socket.io on the existing Express/HTTP server	New
Media	Cloudinary SDK + Multer for image upload	New
Payments	Cashfree (existing)	Existing, unchanged
Email	Nodemailer/Gmail (existing) via the now-wired BullMQ worker	Existing, wired up
Logging	pino (or winston) replacing raw console.log	New
Testing	Vitest + React Testing Library (client); Jest or Vitest + Supertest + mongodb-memory-server (server)	New
Hosting / CI	Vercel (client), Render (API + worker), GitHub Actions	New

7. Feature Overview

Area	Features
Discovery	Home with cuisine categories and featured restaurants; searchable, filterable restaurant listing; restaurant detail with categorised, searchable menu
Ordering	Persistent cart with single-restaurant rule (enforced, matches existing checkout logic); address book; checkout with live price breakdown; Cashfree payment; order confirmation
Tracking	Live order-status timeline (PLACED to DELIVERED) pushed over Socket.IO instead of polling; cancellation while eligible; order history with filters
Account	Profile edit, password change, "become a restaurant owner" request with status visibility
Owner tools	Restaurant profile with cover image; full menu CRUD with image upload, availability toggle, category and price fields; live incoming-order queue with one-tap status advance
Admin tools	Owner-request approval queue; user and restaurant directories with search
Platform	Role-based auth and routing; toasts and skeleton/empty/error states everywhere; responsive 320-1920px; accessible; SEO on public pages; Dockerised backend; CI; structured logs

8. UX and Visual Direction

Concept: "Vivid Kitchen". Where the earlier agency-site brief called for a dark theme, a food-ordering product should feel bright, warm and appetite-driving - the industry standard for a reason. The direction here is a light, high-contrast surface carrying bold, vibrant colour, big food photography, and confident rounded shapes, built with the same rigorous consistency as any professional product.

8.1 Design principles

- **Food is the hero.** Every card built around an image treats the photo as the main event: large, generously cropped, never squeezed.
- **Vibrant but controlled.** One warm primary (tangerine/orange), one fresh secondary (basil green), one punch accent (berry pink) - used deliberately, not everywhere at once.
- **Confident shape language.** Large radii (16 to 28px), soft layered shadows, pill-shaped buttons and filters - friendly and modern, not corporate.
- **One grid, one rhythm** across customer, owner and admin views, so the product never feels like three different apps glued together.
- **Status is always visible.** Order status, cart count, and role are always legible at a glance through colour-coded badges and a persistent cart/status affordance.
- **Motion with purpose.** Micro-interactions on add-to-cart, status changes and card hovers; nothing decorative that gets in the way of ordering food quickly.

8.2 Signature effects (what makes it feel premium)

Effect	Where used
Soft gradient blob backgrounds (warm orange to pink, very low saturation) behind hero and empty states	Home hero, empty cart/orders states
Glass-morphism floating cart bar (blurred white panel, subtle shadow) that appears once items are added	Restaurant menu page, persists across scroll
Animated add-to-cart micro-interaction (button morphs to a checkmark, cart icon bumps)	Menu item cards
Live status timeline with an animated progress line and pulsing "current step" dot	Order tracking page
Card hover lift + image zoom (scale 1.05, 400ms) on restaurant and food cards	Restaurant listing, menu grid
Colour-coded status pills (PLACED, CONFIRMED, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELLED) used identically in customer, owner and admin views	Global
Category pill filters with animated active-state background (layoutId)	Restaurant listing, menu page
Skeleton loaders shaped like the real cards (never generic bars) for restaurants, menu items and orders	Global

8.3 Consistency contract

- Every list page = **PageHeader** + **FilterBar** + content grid/table + pagination, identically structured across customer, owner and admin views.
- One **Button** component with 3 variants (primary, secondary, ghost) and one **StatusPill** component used everywhere a status appears - never a bespoke badge per page.

- All colours, radii, shadows and spacing come from tokens; no hard-coded hex values or ad-hoc paddings in components.
- Every image has a defined aspect ratio and object-fit cover, so grids never jump as images load.
- Owner and admin dashboards reuse the exact same `Table`, `Card` and `EmptyState` components as the customer app, restyled only through tokens, not rebuilt.

9. Data Overview

All 8 collections already exist and are reused unchanged in shape (see LLD section 6 for the exact current schemas): **User**, **Restaurant**, **Food**, **Cart**, **Order**, **Address**, **Payment**, **OwnerRequest**. The only schema-level addition is replacing the plain `image: String` field on **Restaurant** and **Food** with an object holding the Cloudinary URL and public ID, to support proper upload and deletion.

Change	Reason
<code>Food.image / Restaurant.image: string > { url, publicId }</code>	Needed so an image can be replaced or deleted from Cloudinary, not just overwritten by a new URL string
No other schema changes	The existing data model already supports the full v1 feature set; changing it further would only add migration risk

10. Security and Privacy

- Fix the flagged cookie issue: `secure: process.env.NODE_ENV === "production"` and an explicit `sameSite` policy, instead of the current hardcoded `secure: false`.
- Keep the existing JWT access/refresh design; add refresh-token rotation reuse detection if not already present, and ensure logout clears both cookies and the stored refresh token.
- Image uploads limited to jpeg/png/webp, size-capped, scanned for correct MIME type before reaching Cloudinary.
- Socket.IO connections authenticated with the same JWT (from the cookie) at handshake; a user only joins rooms for their own orders, an owner only for their own restaurant's orders.
- Cashfree webhook signature verification (already implemented) is preserved unchanged; do not weaken it while refactoring.
- Remove the unused `razorpay` dependency to reduce attack surface, and keep secrets only in environment variables, validated at boot.

11. Non-Functional Requirements

Area	Target
Performance	Lighthouse mobile ≥ 88 on customer pages; route-level code splitting per role (owner and admin bundles never loaded for a customer); image lists virtualised or paginated, never all loaded at once
Real-time	Order status change reaches the customer and the owning restaurant within 2 seconds via Socket.IO; falls back to a manual refresh if the socket disconnects
Accessibility	WCAG 2.1 AA; keyboard-operable filters, cart, and forms; visible focus states; colour is never the only signal for status (icon + text + colour)
Responsive	320px to 1920px; the owner and admin dashboards are usable on tablet, optimised for desktop
Reliability	Order placement uses a Mongoose transaction so an order is never created without the cart being cleared, or vice versa; payment webhook processing is idempotent (already implemented, keep as-is)

Area	Target
Testability	Server: Jest/Vitest + Supertest + mongodb-memory-server on auth, orders, payments and RBAC. Client: component and flow tests on cart, checkout and order tracking
Operability	Structured JSON logs with request IDs; Dockerised API; health check endpoint (already exists) used by the host

12. Environments and Deployment View

Environment	Frontend	API + worker	Data
Local	Vite dev server, proxy /api and the socket path to :5000	Existing <code>npm run dev</code> plus a new <code>npm run worker</code>	Existing Atlas dev DB, existing Redis
Production	Vercel (static build)	Render web service (API) + Render background worker (BullMQ email worker + cron)	Existing Atlas cluster, existing/managed Redis, Cloudinary, Cashfree, Gmail SMTP

13. Delivery Plan, Risks and Assumptions

Phase	Outcome
P0 Repo & backend hygiene	Git initialised, existing backend committed as-is, then the concrete fixes from LLD section 15 applied and tested
P1 Design system	Tokens, Tailwind config, UI primitives, layout shells for all three roles, dev-only style guide
P2 Real-time + uploads	Socket.IO wired on server and client; Cloudinary upload endpoint and client uploader
P3 Customer app	Home, restaurant listing, menu, cart, checkout, payment, order tracking, history, profile, addresses
P4 Owner dashboard	Restaurant profile, menu manager, live incoming-orders queue
P5 Admin dashboard	Owner requests, users and restaurants directories
P6 Quality	Automated tests (client and server), accessibility and responsive QA, logging cleanup
P7 Ship	Docker, CI, deploy API + worker + client, seed demo data, smoke test, handover note

Risk	Mitigation
Refactoring the existing backend introduces regressions	Fix issues incrementally behind the existing tests-to-be-written; commit the pre-refactor state first so any regression is diffable
Socket.IO adds new failure modes (disconnects, missed events)	Client falls back to polling <code>GET /api/orders/:id</code> if the socket is disconnected for more than a few seconds
Image upload increases Cloudinary/API load	Client-side size/type validation before upload; server-side limits enforced again
Three roles in one SPA increases bundle size	Role-based code splitting so owner/admin code never ships to a customer session
Cashfree integration is fragile to touch	Leave payment controller logic untouched; only reuse it from the new frontend, do not refactor it in this pass

Assumptions

- The backend runs correctly today in local development (as confirmed by the status report) and Cashfree/Cloudinary/SMTP credentials will be supplied by the owner for any new integration.
- A demo dataset (a handful of restaurants and menu items with real-looking photos) is created for development and demos; real restaurant data is out of scope for this pass.
- Delivery fee stays a flat value as today; geolocation-based pricing is a future enhancement.

Definition of done: all three role journeys work end to end against the live deployment; order status updates arrive in real time; every flagged backend issue in the status report is resolved or explicitly deferred with a reason; Lighthouse and accessibility targets are met on the customer app; and the owner receives a working demo login for each role.